Diamond Price Prediction

Executed project report with saved notebook outputs, tables, and charts

Project Description

Diamond Price Prediction

Project Summary

This project is a clean, student-friendly implementation of a diamond price prediction. The main goal is to estimate diamond price from cut, carat, color, clarity, and dimensions. The notebook keeps the workflow simple: load the dataset, understand the columns, clean the data, train a baseline model, and check how well the model performs.

No external repository links or profile links are included in this description. Everything needed to understand and run the project is kept inside this folder.

Problem Statement

A raw dataset is useful only when it is converted into a clear decision-making workflow. In this project, the data is processed step by step so that important patterns can be found and a machine learning model can be trained. The expected output is based on price.

Files Included

File	Purpose	Quick Note
diamonds.csv	Main data source for this folder	2000 sampled rows x 10 columns
Notebook file	Complete Python workflow	Includes loading, cleaning, training, and evaluation
PDF report	Human-readable project summary	Matches the updated project structure

Important Columns

Column	Role
carat	Input feature used in the modelling workflow
cut	Input feature used in the modelling workflow
color	Input feature used in the modelling workflow
clarity	Input feature used in the modelling workflow
depth	Input feature used in the modelling workflow
table	Input feature used in the modelling workflow
price	Target / output column used in the modelling workflow
x	Input feature used in the modelling workflow
y	Input feature used in the modelling workflow
z	Input feature used in the modelling workflow

Workflow Followed

1. Load the data from the local dataset file present in the same project folder.
2. Inspect the structure using shape, column names, missing values, and basic statistics.
3. Clean the dataset by removing empty columns, handling missing values, and keeping only useful features.
4. Prepare the model input by separating features and the target column.
5. Train a baseline model using a simple scikit-learn pipeline with preprocessing.
6. Evaluate the result using practical metrics such as accuracy, classification report, MAE, RMSE, or R2 score depending on the target type.
7. Write observations in a short and direct way so the project can be explained easily.

How to Run

1. Open the notebook in Jupyter Notebook, JupyterLab, Google Colab, or VS Code.
2. Keep the dataset files in the same folder as the notebook.
3. Run the cells from top to bottom.
4. Read the printed metrics and notes at the end before making conclusions.

Final Note

This version keeps the logic understandable and avoids unnecessary complexity. The aim is not to make the notebook look overloaded, but to make it readable, explainable, and useful for a student-level data science portfolio.

Executed Notebook Outputs

The outputs below were generated from the saved notebook after running the code cells in this project folder.

Cell 2 - 2. Load the dataset

```
Dataset shape: (50000, 10)
```

```
carat cut color clarity depth table price x y \
0 0.2220 Ideal E SI2 61.8650 55.1742 375 3.9738 3.8839
1 0.2062 Premium E SI1 59.5220 59.3456 319 3.9334 4.0366
2 0.2194 Good E VS1 55.9346 64.9188 296 4.0829 4.0838
3 0.2832 Premium I VS2 62.8814 57.2678 366 4.2131 4.1285
4 0.3185 Good J SI2 61.7532 58.6792 347 4.3282 4.4422

z
0 2.4018
1 2.3426
2 2.3116
3 2.5996
4 2.7584
```

Cell 3 - 3. Understand the data

```
Rows: 50000
Columns: 10
```

```
column dtype missing_values unique_values
0 carat float64 0 13607
1 cut object 0 5
2 color object 0 7
3 clarity object 0 8
4 depth float64 0 34616
5 table float64 0 37954
6 price int64 0 12254
7 x float64 0 27616
8 y float64 0 27456
9 z float64 0 21059
```

Cell 4 - 4. Clean the data

```
Shape after simple cleaning: (50000, 10)
```

```
carat cut color clarity depth table price x y \
0 0.2220 Ideal E SI2 61.8650 55.1742 375 3.9738 3.8839
1 0.2062 Premium E SI1 59.5220 59.3456 319 3.9334 4.0366
2 0.2194 Good E VS1 55.9346 64.9188 296 4.0829 4.0838
3 0.2832 Premium I VS2 62.8814 57.2678 366 4.2131 4.1285
4 0.3185 Good J SI2 61.7532 58.6792 347 4.3282 4.4422

z
0 2.4018
1 2.3426
2 2.3116
3 2.5996
4 2.7584
```

Cell 5 - 5. Select the target column

```
Selected target column: price
```

```
0 375
1 319
2 296
3 366
4 347
Name: price, dtype: int64
```

Cell 6 - 6. Quick EDA

```
Numeric columns: 7
Categorical columns: 3
No missing values found in the previewed columns.
```

```
count mean std min 25% 50% \
carat 50000.0 0.799343 0.475320 0.1979 0.397400 0.70530
depth 50000.0 61.746225 1.806641 43.1156 60.639100 61.77490
table 50000.0 57.453482 2.451491 41.6400 55.745350 57.25275
price 50000.0 3944.791580 3998.117527 292.0000 950.000000 2418.00000
x 50000.0 5.735036 1.128075 0.0000 4.704000 5.68135
y 50000.0 5.737388 1.148788 0.0000 4.710550 5.69180
z 50000.0 3.541472 0.710656 0.0000 2.903975 3.50880

75% max
carat 1.047000 4.8269
depth 62.878550 78.9465
table 58.956200 99.4730
price 5351.000000 18844.0000
x 6.553325 10.6434
y 6.549600 57.4898
z 4.049900 32.8457
```

Cell 7 - 7. Prepare features and target

```
Using 1200 sampled rows for model training speed.
Problem type: regression
Feature shape: (1200, 9)
Target unique values: 1067
```

Cell 8 - 8. Train a baseline model

```
Training completed.
```

Cell 9 - 9. Evaluate the model

```
MAE : 774.3471
RMSE: 1476.2407
R2 : 0.8557
```